



Offline AI Vision Assistant

Technical Summary & Architecture Overview

Version 1.0 | Android (arm64-v8a) | minSdk 26 | targetSdk 34
Kotlin 2.3.0 | LiteRT-LM 0.16.1 | Gemma 3n E2B int4 | ML Kit OCR

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Architecture
4. Core Components
5. VLM Engine (Gemma 3n E2B)
6. ML Kit OCR Pipeline
7. Camera Pipeline
8. Speech Recognition & TTS
9. Adaptive Memory System
10. Performance Optimizations
11. Safety & Stability
12. Tech Stack
13. File Structure

1. Executive Summary

Vyze is a fully offline AI vision assistant for Android that helps blind and visually impaired users understand their surroundings in real time. Powered by Gemma 3n E2B (2B parameter multimodal model) running on-device via Google's LiteRT-LM framework with NPU/GPU acceleration.

Users interact hands-free through continuous speech recognition: they speak a question like "What is in front of me?" or "Read this label", and Vyze captures a camera frame, runs VLM inference (or fast ML Kit OCR for text queries), and speaks the result aloud via sentence-buffered TTS streaming.

Key differentiators: (1) 100% offline -- no data leaves the device, (2) real-time voice-first UX with barge-in interruption, (3) adaptive memory that learns from past interactions, (4) hybrid OCR pipeline (ML Kit + Gemma) for 300ms text extraction, (5) user-driven language mirroring (auto-detect + respond in user's language), and (6) aggressive latency optimizations achieving sub-2-second time-to-first-audio.

2. Problem Statement

Over 2.2 billion people globally have vision impairments (WHO, 2023). Existing assistive technology relies heavily on cloud-based AI services that require internet connectivity, introduce latency (1-3 seconds per query), raise privacy concerns by uploading camera images, and fail in offline environments (rural areas, subways, international travel).

Vyze addresses these gaps by running the entire vision-language pipeline locally on the user's phone. The Gemma 3n E2B model (3.66 GB int4 quantized) fits within mobile GPU memory constraints while providing high-quality scene understanding, text extraction, and multi-language support.

3. Solution Architecture

Vyze follows a modular architecture with clear separation of concerns:

- Camera Layer: CameraX ImageAnalysis delivers frames to an in-memory buffer (zero disk I/O)
- ML Kit OCR: Fast on-device text extraction (80-150ms) for text queries
- VLM Engine: LiteRT-LM + Gemma 3n E2B with NPU/GPU fallback
- Prompt Builder: Dynamic prompt system with navigation, direct-query, continuous-mode, and language mirror rules
- TTS Manager: Google neural TTS with sentence-buffered streaming and language-aware voice switching
- Speech Recognizer: Android SpeechRecognizer with aggressive silence endpoints + language detection
- Adaptive Memory: Room database with vector similarity search for contextual recall
- Core Controller: Orchestrates the full pipeline with session isolation and state management

Data Flow

User speaks "What is this?"

- > `SpeechRecognizer.onResults()` extracts text + detected language
- > `CameraFragment` updates TTS voice to match detected language
- > `VyzeCoreController.setUserLocale()` switches TTS + prompt language
- > `CameraSetupDelegate.takeSnapshot()` copies fresh `Bitmap`
- > `VyzeCoreController.triggerSnapshot()` routes through OCR or VLM:
 - TEXT QUERY: ML Kit OCR (80-150ms) -> Gemma interprets
 - SCENE QUERY: Gemma 3n E2B direct inference
- > `onTokenGenerated()` streams tokens into `sentenceBuffer`
- > `flushSentenceBufferIfReady()` sends clause-level chunks to TTS
- > TTS speaks in user's detected language with matching voice
- > `onDone()` resets to IDLE + restarts `SpeechRecognizer`

4. Core Components

4.1 VyzeCoreController

The central orchestrator that manages the full capture-to-speech pipeline. Routes text queries through ML Kit OCR first, then feeds clean text to Gemma for interpretation. Manages session isolation, sentence streaming, and language mirroring.

- Session isolation via UUID-based `activeSessionId` (prevents stale response leakage)
- OCR-first routing: ML Kit for text queries, Gemma for scene queries
- Language mirroring: stores `activeUserLocale`, passes to prompt builder + TTS
- Coroutine-based inference with Job cancellation on new queries
- Sentence-buffered TTS streaming with clause-boundary flushing
- Duplicate detection via `lastDescribedObject` cache with 4s debounce
- 15-second watchdog timer to prevent indefinite ANALYZING state

4.2 VlmEngineManager

Wrapper around Google's LiteRT-LM framework that manages the Gemma 3n E2B engine lifecycle. Provides NPU/GPU multi-backend fallback and the `analyzeImage()` API.

- Multi-backend fallback: NPU (preferred) -> GPU (fallback) -> Error
- Model resolution from `/storage/emulated/0/Download/` or `getExternalFilesDir(null)`
- Dynamic bitmap preprocessing: 256px (scene) or 384px (text) center-cropped squares
- Gemma turn-format prompting with compact system directives
- Greedy decoding: `maxTokens=35`, `temperature=0.1`, `topK=1`
- JPEG quality 75 with pre-allocated 8KB `ByteArrayOutputStream`
- GPU warm-up: dummy 1x1 image pre-compiles OpenCL/Vulkan kernels
- Non-destructive interrupt: `latch.countDown()` without `engine.close()`

4.3 OcrHelper

ML Kit on-device OCR wrapper supporting Latin and Chinese scripts. Runs 10-30x faster than full VLM inference for text extraction.

- Latin script: English, Malay, Indonesian, Vietnamese, European languages (~5MB)
- Chinese script: Simplified + Traditional Chinese (~20MB)
- Auto-detects script in image, no manual routing needed
- Confidence scoring for fallback decisions
- Falls back to Gemma for handwriting and complex layouts

4.4 CameraSetupDelegate

Manages CameraX use cases and provides zero-disk-I/O frame extraction.

- ImageAnalysis.Analyzer decodes YUV_888 to ARGB_8888 at camera framerate
- Atomic frame swap with frameLock prevents recycling during extraction
- Frame counter guarantees fresh frames on voice-triggered snapshots
- takeSnapshot() polls for post-query frame with 300ms timeout + fallback

5. VLM Engine -- Gemma 3n E2B

The vision-language model is Google's Gemma 3n E2B (Edge 2 Billion parameters), int4 quantized to 3.66 GB. It runs entirely on-device via the LiteRT-LM framework with NPU/GPU acceleration.

Model Specifications

Model:	gemma-3n-E2B-it-int4.litertlm
Size:	3.66 GB (int4 quantized)
Parameters:	~2 billion
Backend:	NPU (preferred) -> GPU (fallback)
Framework:	LiteRT-LM 0.16.1
Max Output Tokens:	35 (greedy)
Temperature:	0.1
Top-K:	1
Image Input:	256x256 or 384x384 center-cropped square
Prompt Format:	Gemma turn system (<start_of_turn>)

Inference Pipeline

1. Detect query type (text vs scene)
2. If text query: run ML Kit OCR first (80-150ms)
3. Center-crop bitmap to 1:1 square (preserves spatial alignment)
4. Scale to 256px (scene) or 384px (text extraction)
5. JPEG encode at quality=75 (pre-allocated 8KB buffer)
6. Build compact prompt with language mirror directive
7. Inject OCR text if available (skips character-level reading)
8. Create Conversation with SamplerConfig(topK=1, temp=0.1)
9. Engine.sendMessageAsync(contents, callback)
10. Stream tokens via onTokenGenerated callback
11. Sentence-buffered flush to TTS at clause boundaries

6. ML Kit OCR Pipeline

Vyze uses a hybrid OCR approach: ML Kit for fast text extraction, Gemma for intelligent interpretation. This achieves 10-30x faster text recognition compared to using the VLM alone.

How It Works

```
Text query arrives ("Read this label")
-> isTextExtractionQuery() detects keyword
-> ML Kit OCR runs on bitmap (80-150ms)
    Latin script first, then Chinese if nothing found
-> OCR text injected into prompt: "OCR: Diclac Retard..."
-> Gemma receives clean text + image
-> Interprets immediately (skips character-level reading)
-> TTS speaks result (~300-500ms total)
```

Supported Scripts

Latin:	English, Malay, Indonesian, Vietnamese, Turkish, European (~5MB)
Chinese:	Simplified + Traditional Chinese (~20MB)
Total overhead:	~25MB added to APK

Fallback Behavior

- ML Kit returns good text -> Gemma interprets with context
- ML Kit returns nothing -> checks for handwriting -> falls back to Gemma
- Gemma handles handwriting, complex layouts, spatial context

7. Camera Pipeline

Vyze uses CameraX with ImageAnalysis for zero-disk-I/O frame extraction. All processing happens in memory -- no takePicture, no JPEG decode from disk.

Frame Extraction Flow

- ImageAnalysis.Analyzer runs on single-thread ExecutorService
- YUV_888 -> ARGB_8888 conversion with rotation correction
- New frame swapped into AtomicReference via getAndSet()
- Frame counter incremented to signal fresh frame availability

Snapshot Capture

- takeSnapshot() records frameCounter at query time
- Polls for frameCounter > counterAtQuery (guarantees fresh frame)
- 300ms timeout with fallback to latestFrame.get()
- Deep copy via Bitmap.copy(ARGB_8888, true) before VLM

8. Speech Recognition & TTS

8.1 Speech Recognition

Uses Android's SpeechRecognizer with aggressive silence endpoints and automatic language detection for fast, hands-free interaction.

- COMPLETE_SILENCE_LENGTH: 400ms (was 1500ms)
- POSSIBLY_COMPLETE_SILENCE_LENGTH: 300ms (was 1500ms)
- MINIMUM_LENGTH: 1000ms
- PARTIAL_RESULTS: true (enables early intent parsing)
- Language detection: EXTRA_LANGUAGE from speech results
- Auto-restart after TTS completes (continuous hands-free loop)

8.2 Text-to-Speech

Neural voice synthesis via Google TTS engine with language-aware voice switching and deterministic completion tracking.

- Engine: Google TTS (com.google.android.tts) for neural voice quality
- Voice selection: Highest QUALITY_HIGH/QUALITY_VERY_HIGH voice for locale
- Language mirroring: Auto-switches TTS voice to match detected spoken language
- Pitch: 0.96f (-2%) for warmer, non-metallic tone
- Rate: 0.98f (-2%) for conversational cadence
- Audio: USAGE_ASSISTANCE_ACCESSIBILITY (non-duckable)
- Volume: KEY_PARAM_VOLUME = 1.0f (stable full volume)
- Audio Focus: AUDIOFOCUS_GAIN_TRANSIENT
- Cadence: Punctuation-aware spacing for natural pauses

8.3 Language Mirroring

Vyze automatically detects the user's spoken language and mirrors it to both Gemma's output and the TTS voice. No hardcoded language lists.

- SpeechRecognizer extracts EXTRA_LANGUAGE from results Bundle
- Locale.forLanguageTag() converts BCP-47 tag to java.util.Locale
- DynamicPromptBuilder adds: "Respond strictly in {language}."
- TTSManager.switchToLocale() selects best neural voice for language
- Falls back to Locale.US if no voice pack installed

8.4 Sentence-Buffered Streaming

Tokens from VLM inference are buffered and flushed at natural clause boundaries for minimal time-to-first-audio.

- First chunk: flush at ≥ 1 word or ≥ 3 characters (instant feedback)
- Subsequent chunks: flush at commas/colons (12+ chars) or terminators (10+ chars)
- Final flush: force-flush all remaining text on onComplete
- Utterance ID tracking via pendingUtteranceIds (deterministic, no polling)

- 400ms post-drain grace period for AudioTrack hardware buffer

9. Adaptive Memory System

Vyze maintains a local Room database that records interactions, user preferences, and environment context to provide increasingly personalized assistance.

Data Schema

```
VyzeMemoryEntity:
- category: String (preference/environment/interaction)
- key: String
- value: String
- metadata: String (query text for interactions)
- timestamp: Long

InteractionRecord:
- id: Long (auto-generated)
- timestamp: Long
- imageEmbedding: FloatArray (vector similarity)
- rawPrompt: String
- generatedOutput: String
- userFeedback: String
```

Contextual Prompt Injection

- Similar past interactions retrieved via vector similarity search
- User preferences injected as compact section in prompt
- Environment context included for spatial continuity
- All queries execute off-main-thread via Dispatchers.IO

10. Performance Optimizations

- Prompt Trimming: Compact system directives: ~80 tokens prefill (was ~875 tokens)
- ML Kit OCR Pre-pass: Text queries: 300-500ms (was 4-6s with VLM only)
- Dynamic Resolution: 256px for scenes, 384px for text (auto-detected by keywords)
- Greedy Decoding: maxTokens=35, temperature=0.1, topK=1 for fastest generation
- First-Chunk Flush: 1 word / 3 chars triggers first TTS audio (~200ms)
- GPU Warm-up: Dummy 1x1 image pre-compiles OpenCL/Vulkan kernels on init
- JPEG Optimization: Quality 75 + pre-allocated 8KB buffer (15-30ms faster)
- Zero Disk I/O: In-memory Bitmap buffer from ImageAnalysis
- Frame Counter: Guarantees fresh frame on every voice query
- Engine Interrupt: latch.countDown() releases await without destroying engine
- Session Isolation: UUID-based sessionId prevents stale callback leakage
- NPU/GPU Fallback: Auto-tries NPU first, falls back to GPU
- Language Mirroring: Auto-detect + respond in user's language
- Barge-In: Instant TTS stop on touch/speech input
- Watchdog Timer: 15s timeout prevents indefinite ANALYZING state

11. Safety & Stability

State Machine

```
IDLE -> (tap/speech) -> ANALYZING -> (VLM complete) -> SPEAKING -> (TTS done) -> IDLE
IDLE -> (barge-in) -> ANALYZING (cancels previous + stops TTS)
ANALYZING -> (15s timeout) -> IDLE + error speech
```

Crash Prevention

- 15-second watchdog timer forces ANALYZING -> IDLE if inference hangs
- Bitmap.isRecycled + getPixel(0,0) corruption check before VLM
- Session ID gating on all callbacks (onTokenGenerated, onComplete, onError)
- Coroutine Job cancellation with isActive checks at 3 points
- Non-destructive interrupt: engine stays alive for next query
- 1-second debounce on trigger events prevents double-fire

Memory Management

- Bitmap.recycle() in finally blocks for original and scaled bitmaps
- Conversation lifecycle: manual close() (not use{}) to prevent SIGSEGV
- Coroutines 1.10.1 (fixed SendChannel.close\$default crash with Kotlin 2.3.0)

12. Technology Stack

Languages & Frameworks

Language:	Kotlin 2.3.0
Build System:	Gradle 8.14.3 + Kotlin DSL
Min SDK:	26 (Android 8.0)
Target SDK:	34 (Android 14)
Architecture:	ARM64-v8a only (64-bit GPU delegates)

AI & ML

VLM Model:	Gemma 3n E2B int4 (3.66 GB)
Inference:	LiteRT-LM 0.16.1 (Google)
Backend:	NPU -> GPU fallback chain
OCR:	ML Kit Text Recognition (Latin + Chinese)
OCR Size:	~25MB total (5MB Latin + 20MB Chinese)

Android Libraries

Camera:	CameraX 1.3.1
Database:	Room 2.7.0
Navigation:	Navigation 2.7.7
Coroutines:	kotlinx-coroutines 1.10.1
Lifecycle:	Lifecycle 2.7.0
UI:	Material 1.11.0 + ConstraintLayout 2.1.4

Build & Release

Release Build:	R8 minification + resource shrinking
JNI Packaging:	useLegacyPackaging = true (uncompressed .so)
NDK Filters:	arm64-v8a only

13. Key File Structure

<code>VyzeApplication.kt</code>	App singleton, dependency injection, global error handler
<code>MainActivity.kt</code>	Speech recognition, language detection, TTS orchestration
<code>VyzeCoreController.kt</code>	Pipeline orchestrator, OCR routing, session management
<code>VlmEngineManager.kt</code>	LiteRT-LM wrapper, NPU/GPU fallback, Gemma engine lifecycle
<code>DynamicPromptBuilder.kt</code>	Prompt assembly with language mirror + intent-based rules
<code>OcrHelper.kt</code>	ML Kit OCR wrapper (Latin + Chinese scripts)
<code>CameraSetupDelegate.kt</code>	CameraX setup, frame buffer, bitmap extraction
<code>CameraFragment.kt</code>	Camera UI, voice triggers, continuous mode, state machine
<code>TTSManager.kt</code>	Neural TTS, voice switching, utterance tracking, audio focus
<code>MemoryRepository.kt</code>	Adaptive memory, vector search, interaction records
<code>VyzeDatabase.kt</code>	Room database, DAOs, entities